

Sujet commun SAÉ 2.01-SAÉ 2.02

Description du monde à simuler

Comme pour les SAÉ 1.01 & SAÉ 1.02, les deux SAÉ partageront un noyau commun.

Ce noyau consiste en la modélisation et la simulation de robots mineurs exploitant un nouveau monde.

Principes

Il y aura dans ce monde des mines ; chaque mine contient un nombre (entre 50 et 100) de minerais d'une seule nature : soit de nickel (1 ou 2 mines), soit d'or (1 ou 2 mines). Deux entrepôts permettent de stocker les minerais récoltés : l'un permettant de stocker l'or, l'autre le nickel. Des robots (au moins un de chaque type et au plus 5), chacun étant spécialisés dans l'extraction de minerai soit de nickel ou d'or, exploreront le monde afin d'aller dans les mines, extraire des minerais et les amener dans les entrepôts. Chaque robot a une capacité de stockage de minerais (entre 5 et 9) qui lui est propre, et une capacité (entre 1 et 3) d'extraction de minerais (nombre de minerais récoltés en une seule action d'extraction).

Le monde est décomposé en une matrice de 100 secteurs (10 x 10). Chaque secteur du monde sera soit un terrain (au moins 90), soit un plan d'eau.

Les robots peuvent aller sur un terrain du moment qu'il n'est pas occupé par un autre robot.

Les robots ne peuvent pas aller sur les plans d'eau.

Sur les terrains il peut y avoir soit rien, soit un entrepôt, soit une mine. On ne peut pas avoir un entrepôt et une mine sur un même secteur.

Les robots peuvent :

- Avancer dans une direction : Nord, Est, Sud ou Ouest,
- Récolter des minerais s'ils sont dans une mine de leur spécialité et s'il reste au moins un minerai. Le robot récolte, s'il y a suffisamment de minerai, autant que sa capacité d'extraction, sinon il récolte ce qu'il reste.
- Déposer ses minerais s'il est dans l'entrepôt dédié au minerai dont il est spécialisé.

Chaque robot a un numéro unique, chaque mine a un numéro unique, chaque entrepôt a un numéro unique.

À chaque tour, chaque robot peut réaliser au plus une action valide.

Informations à visualiser

Visuellement un secteur sera représenté par une matrice 2x2, chaque case pouvant avoir un caractère.

Secteur vide :

Secteur représentant un plan d'eau :

X	X
X	X

Secteur possédant une mine : la mine est identifiée sur la première ligne :
ici il s'agit de la mine n°3

M	3

Secteur possédant un entrepôt, l'entrepôt est identifié sur la première ligne :
ici il s'agit de l'entrepôt n°1

E	1

Secteur possédant un robot, celui-ci est identifié sur la deuxième ligne :
ici le robot n°2

R	2

Un secteur peut donc être occupé par un entrepôt ou une mine ET SIMULTANÉMENT par un robot :

M	1
R	1

 ; ici dans la Mine 1 se trouve le robot n°1.

La représentation du monde pourra se faire de la façon suivante (à noter que l'affichage pourra, dans un premier temps, se faire en mode console) :

	0	1	2	3	4	5	6	7	8	9	
0		X	X						E	2	
1		X	X					R	2		
2						X	X			M	3
3				X	X						
4				X	X						
5									M	2	
6							X	X			
7							X	X			
8			E	1	X	X					
9							M	1			
						R	1				

tour	75
M1	9 5 OR 72 / 90
M2	5 8 NI 55 / 75
M3	2 9 OR 0 / 80
E1	8 2 NI 20
E2	0 8 OR 96
R1	9 5 OR 2 / 5
R2	7 3 NI 0 / 7

On affichera le numéro du tour actuel et les informations concernant toutes les mines, robots et entrepôts

Pour les mines : M suivi de son n°, puis sa ligne et sa colonne, son type, son nombre de minerais actuel et sa capacité initiale.

Pour les entrepôts : E suivi de son n°, puis sa ligne et sa colonne, son type, son nombre de minerais actuel.

Pour les robots : R suivi de son n°, puis sa ligne et sa colonne, son type, son nombre de minerais actuel et sa capacité de stockage.

Astuce : pour l'affichage dans la console, il peut être plus facile de remplir une matrice de caractères avec les informations à afficher puis d'afficher cette matrice de caractères.

Contraintes à respecter :

- Utiliser l'EDI IntelliJ avec Open-JDK 21 et Maven
- Utiliser GITHUB dans IntelliJ
- Mettre les enseignants (P. POLET, C. ROZE et J. VION) dans les membres du projet GITHUB
- Constitution des équipes : 4 à 5 étudiants du même groupe de TP : envoyer la constitution des équipes avant la fin de la première séance à P. POLET, C. ROZE et J. VION.

Livrables pour la SAÉ2.01 :

Dans le cadre de la SAÉ2.01, c'est l'utilisateur qui pilote chaque robot.

À chaque tour, pour chaque robot, l'utilisateur indique l'action à réaliser.

Livrable 1.01 : Diagrammes de classes

Livrable 1.02 : Création du projet sur git-hub (y inscrire Philippe Polet, Christelle Rozé et Julien Vion). Rapport des 1er commits.

Livrable 1.03 : Maquettage des interactions pour la saisie des actions à réaliser par l'utilisateur (version pdf du maquettage requis)

Livrable 1.04 : Rapport de test automatisé JUnit pour au moins deux des principales méthodes (fin mai). Le rapport doit contenir (a) le jeu de test (tableau de test tel que réalisé dans R2.03) avec l'indication des critères de test qui ont été utilisés, (2) un rapport du résultat du test de l'application (par exemple, une capture d'écran du résultat de l'exécution).

Livrable 1.05 : Code java final 2 versions : 1 en mode console, et 1 intégrant les interactions utilisateur graphiques. + exécutable sous forme de JAR

Attention : Organisez-vous pour utiliser GIT tout au long du projet. Nous attendons différents commits de la part de différentes personnes de l'équipe.

Nous vous conseillons d'avancer sur le projet avec cette organisation :

- 1ère séance en autonomie : Constitution groupes, création UML et début code Java (console)
- 2ème séance en autonomie : Création du maquettage (interaction de l'utilisateur avec une interface graphique pour indiquer les actions des robots à chaque tour).
- 3ème séance en autonomie : Codage de la partie commune et des interactions en console
- 4ème séance en autonomie : codage de la partie IHM et correction de code.
- 5ème séance en autonomie : finalisation de la partie IHM + Tests (JUnit)

- 6^{ème} séance en autonomie : Finalisations des tests (JUnit et test de l'application) et de l'ensemble des livrables.

L'ensemble des dépôts devra être fait la veille du DS.

Livrables pour la SAÉ2.02 :

A venir